Introduction to Robotics: Subsumption Architecture

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

Introduction

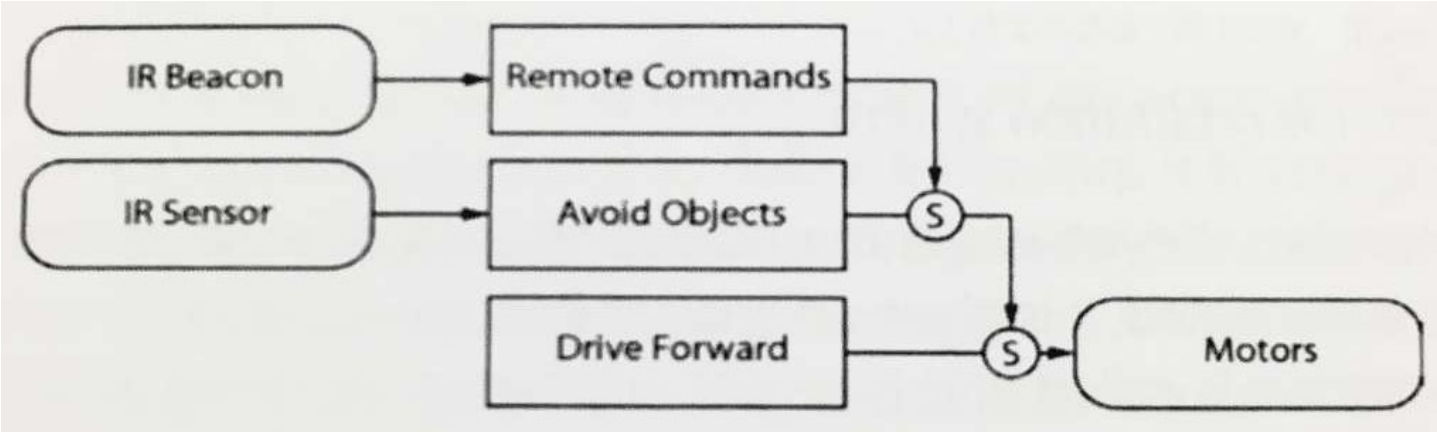
- We have learned multi-thread programming and basic scheduling algorithms
  - How to assign priorities to threads?
  - How to schedule based on priority?
  - With thread safe, how to arrange the execution orders?
- We will learn basics about behavior-based robotics
  - A practical implementation: subsumption architecture

Subsumption Architecture 1

- For many new programmers, they would like to use a series of if-then statements
  - Very easy to start, but hard to maintain: tangled up and difficult to expand
- How to make your program extendable?
  - Get some inspirations from Object-Oriented Programing and Aspect-Oriented Programing
    - Objects or aspects or behaviors can be nicely encapsulated with an easily-understood structure

Subsumption Architecture 2

- We can use the obstacle avoidance as an example
  - Drive forward
  - If too close to an object, change direction
  - Allow user to take control with IR remote



- How to define the actions and their order?

Subsumption Architecture 3

- Behavior
  - Behaviors serve as the building blocks for robotic actions
  - Behaviors are independent and computational entities and operate concurrently
    - The overall behavior is emergent: there no explicit "controller" determines what will be done or makes calls from a function to another function – behaviors are independent!
    - There may be a coordinated control program in the schema of a behavior, but there is no external controller of the behaviors – information in those behaviors determine which behavior will be in action next, and the coordination program just make a help to switch a behavior to the context
  - Behavior can be simple, e.g. moving forward, or can be complex, e.g. mapping for a room.

Subsumption Architecture 4

- Priority
  - There is external controller, instead the order of the behaviors is maintained by priorities
  - For the “released” behaviors will be coordinated based on the priorities: the behavior with higher priority will be executed first
    - Other behaviors with lower priority need to wait till the current behavior finish (into yield status)
    - Or if a new behavior with higher priority is released, the current behavior will halt (or be gracefully switched off), and give way to the new behavior
  - The priority, “release”, actions and “finish” of a behavior should be defined in its definition

Subsumption Architecture 5

- How to give definitions to priority, actions, “release” and “finish” in programs?
- In leJOS, there is a Behavior interface
  - *lejos.robotics.subsumption.Behavior*
    - To define a behavior, we need to implement the Behavior interface by giving implementations to the following methods in it:
    - *boolean takeControl()*
    - *void action()*
    - *void suppress()*
  - *lejos.robotics.subsumption.Arbitrator*
    - The behaviors defined by implementing Behavior interface can be added to the **coordinate program** of *Arbitrator*

Subsumption Architecture 6

- How to give definitions to priority, actions, "release" and "finish" in programs? (cont'd)

  `lejos.robotics.subsumption.Behavior`

  - The "release"
    - `boolean takeControl()`
    - E.g. return (distance < 40); // take control when distance is smaller than 40
  - The actions
    - `void action()`
    - When take control, do the logic of the behavior, e.g. take a detour to avoid the obstacle

Subsumption Architecture 7

- How to give definitions to priority, actions, "release" and "finish" in programs? (cont'd)

  lejos.robotics.subsumption.Behavior

  - The "finish"
    - *void suppress()*
    - It tells when to terminate the behavior actions, e.g. there is a *continue* flag in the action, if it's *true*, then repeat the actions, then in *suppress()* method, you can set *continue* flag as *false*
  - The priority
    - We will use another class: *Arbitrator*

Subsumption Architecture 8

- How to give definitions to priority, actions, "release" and "finish" in programs? (cont'd)

  `lejos.robotics.subsumption.Arbitrator`

  - The behaviors defined by implementing Behavior interface can be added to the coordinate program of `Arbitrator`
  - We need to use the methods in this class as:
    - Constructor: `public Arbitrator(Behavior [] behaviors)`
    - Starter: `public void go()`
      - In the textbook, it's `public void start()`. We should use `public void go()`.
    - Stopper: `public void stop()`

Subsumption Architecture 9

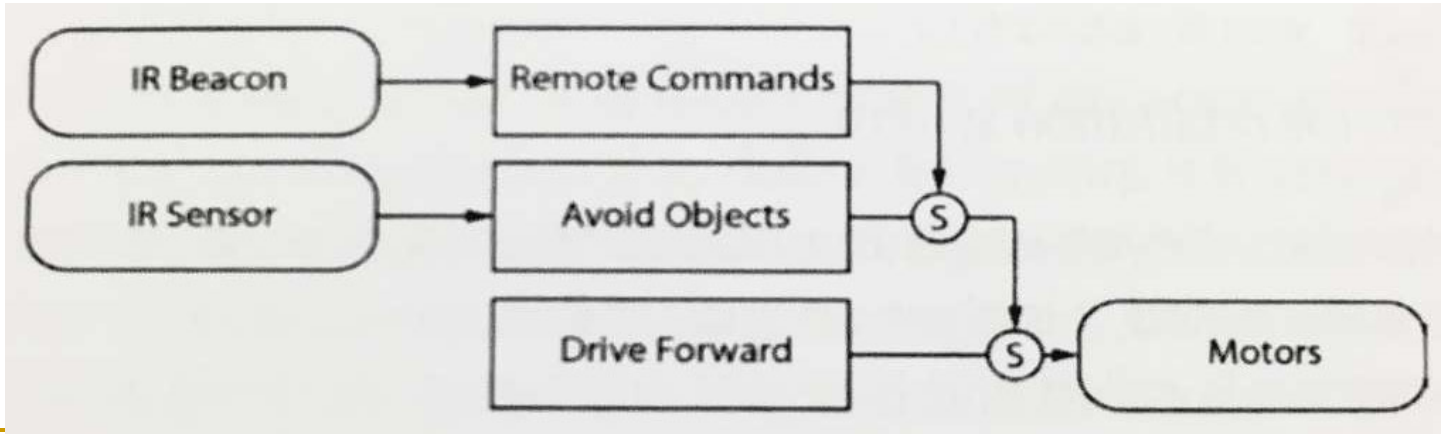
- How to give definitions to priority, actions, "release" and "finish" in programs? (cont'd)

  lejos.robotics.subsumption.Arbitrator

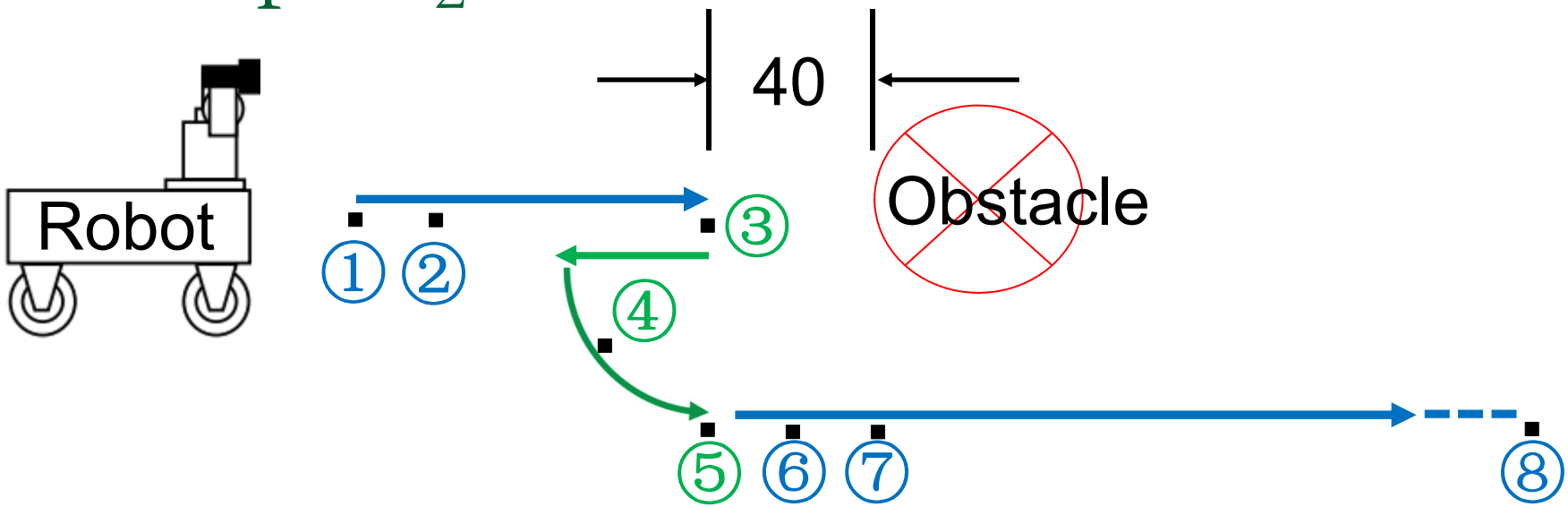
  - The priorities are specified by the order of elements in the Behavior array inside *public Arbitrator(Behavior [] behaviors)*
    - The higher the array index number for a Behavior, the higher the priority level
    - There is not much code needed here to coordinate the behaviors, as long as they are well defined
      - *i.e. boolean takeControl(), void action(),* and *void suppress()* are clearly defined
    - We will take a look on the example code

Examples 1

- In the obstacle avoidance example:
  - BehaviorForward.java
  - BehaviorProximity.java
  - BehaviorRemote.java
  - BehaviorMain.java
  - Page 273 – 278 in the textbook



Examples 2



Arbitrator

Blue: BehaviorForward, Green: BehaviorProximity

Timeline - the time when each method is called:

(Who makes the calls? Arbitrator. How? Page in the end.)

① takeControl	③ takeControl	⑥ takeControl
② action	④ action	⑦ action
	⑤ suppress	⑧ suppress

Examples 3

- BehaviorForward.java
  - When implementing an interface in java, you need to override the abstract methods

```
@Override
public boolean takeControl() {
        return true;
}
```

    - Can always take over if no other behavior with high priority
  - Once the behavior take control, we can take the action

```
@Override
public void action() {
        left.backward();
        right.backward();
}
```

Examples 4

- BehaviorForward.java (cont'd)
  - Always run, no need to stop

```
@Override
public void suppress() {
        // nothing to suppress
}
```

- BehaviorProximity.java
  - When to "release"? When the object is too close:

```
@Override
public boolean takeControl() {
        return (ir.distance < 50);
}
```

Examples 5

- BehaviorProximity.java (cont'd)
  - Once this behavior takes control, we drawback and make a turn

```
@Override
public void action() {
        backing_up = true;

        left.rotate(600, true);
        right.rotate(600);

        left.rotate(-450, true);
        right.rotate(450);

        backing_up = false;
}
```

Examples 6

- BehaviorProximity.java (cont'd)
  - To tell how to gracefully terminate the behavior, we use the *`public void suppress()`* method

```
@Override
public void suppress() {
        // Wait until backup done
        while(backing_up) {Thread.yield();}
}
```

  - Here if the coordinator calls, to let this behavior stop, actually this behavior won't stop immediately, instead it will let the coordinator wait till the boolean variable **`backing_up`** is set to be **`false`** in `action()` method (finish the job and then terminate)
    - Compare with previous example: behavior tells how to stop

- BehaviorRemote.java: Not very special

Examples 7

- BehaviorMain.java

It's very easy here: just add the behaviors in the constructor

```
Behavior b1 = new BehaviorForward(left, right);
Behavior b2 = new BehaviorProximity(left,   right, ir);
Behavior b3 = new BehaviorRemote(left, right, ir);
Behavior [] behave = {b1, b2, b3};
arby = new Arbitrator(behave);
arby.go();
```

- `arby.go();` starts the arbitration of Behaviors and runs an endless loop.
- The `arby.start();` line in the textbook should be updated as `arby.go();`
- You can use button press as the user signal and then call the `public void stop()` method to terminate Arbitrator.

Question

- What's the mechanism that these behaviors can be "automatically" switched?
  - Secret in Arbitrator class?
  - Take a look at the code of Arbitrator class:
    "LeJOS EV3 EV3 Runtime" library in eclipse under your EV3 project
    -> ev3classes.jar
    -> lejos.robotics.subsumption
    -> Arbitrator.class
    look at `private class Monitor extends Thread` and `public void go()`
  - The key is multi-thread programming!
  - This has been implemented, you don't need to worry about it any more.